
LICENCIATURA EN CIENCIAS DE LA COMPUTACIÓN

Compiladores

Descripción del Lenguaje `tinyS`

Ana Carolina Olivera
Universidad Nacional de Cuyo
Facultad de Ingeniería

Primer Semestre 2026

Índice general

1 Introducción

Como proyecto final de la asignatura Compiladores tendrán la oportunidad de desarrollar un compilador completo para un lenguaje de programación de alto nivel al que llamaremos **tinyS**. El objetivo detrás de este proyecto final es que apliquen los conocimientos vistos durante estos años de universidad. Por este motivo, no solo utilizarán conceptos de Lenguajes Formales y Computabilidad sino que también requerirán de habilidades y conocimientos de las asignaturas Paradigmas de Programación, Ingeniería de Software 1, Ingeniería de Software 2, entre otras. Aunque es un lenguaje académico incluye características propias de los lenguajes de programación modernos.

Todo el código de **tinyS** está organizado en clases e implementaciones. **tinyS** es **type safe**: se debe garantizar que los métodos sean aplicados a datos de tipo correcto. Mientras que el tipado estático impone una disciplina fuerte de programación en **tinyS**, también garantiza que no ocurran errores de tipo en tiempo de ejecución. **tinyS** es fuertemente tipado.

En las secciones siguientes se presentan las particularidades de **tinyS** que deberán tener en cuenta a la hora de implementar un compilador para este lenguaje.

Antes de empezar

Un código fuente a compilar en **tinyS** estará compuesto por un único archivo. El compilador de **tinyS** compila un programa de la siguiente manera:

```
java -jar tinyS.jar archivo_entrada.s [archivo_salida]
```

Un compilador para **tinyS** deberá compilar el archivo **archivo_entrada.s** siguiendo exactamente el formato anterior. El nombre del archivo de salida es opcional y deberá generarse en el mismo lugar donde se encuentra el archivo de entrada. Si no se proporciona nombre para el archivo de salida el mismo es por defecto **archivo_entrada.asm**.

Los mensajes de error que el compilador genera cuando encuentra un error léxico, sintáctico o semántico **no deben contener acentos** para evitar problemas de incompatibilidades y no utilizar caracteres poco frecuentes. El compilador debe ser capaz de ejecutarse correctamente en **cualquier sistema operativo actual**.

La lectura y comprensión de este documento son obligatorias para poder desarrollar satisfactoriamente el proyecto final. Tomarse el tiempo para leer y entender el contenido de este documento es parte de la evaluación.

2 Declaraciones

2.1. Clases: `class`

Todo el código de `tinyS` está organizado en clases e implementaciones, juntas representan a las clases en el paradigma orientado a objetos. Múltiples `class` pueden ser definidas en el mismo archivo. La definición de una clase tiene la siguiente forma:

```
Ejemplo 1. class <Idclass> [ : <Idclass> ] {  
    <Atributos>*  
}  
...  
impl <Idclass> {  
    <Métodos>*  
}
```

Todos los nombres de las clases son visibles globalmente (no hay clases privadas). Los nombres de clases deben **comenzar** con una **letra Mayúscula** y terminar con una **letra, dígito o guión bajo**. Salvo el método que se invoca obligatoriamente al crear una instancia de una `class`. Dicho método será identificado por tener como nombre un `.` únicamente como nombre.

Puede haber **más** de un `class` para un `idclass` con el mismo nombre en el mismo código fuente, tampoco existe restricción en el número de `impl` para el mismo `idclass` en el mismo código fuente. Al menos debe haber un `impl` para cada `class` dentro del código y uno de los `impl` para un `idclass` debe tener al constructor.

2.2. El método `start`

Cada código fuente debe tener un método `start` que no tome parámetros formales. El método `start` debe estar definido al final de todas las `class`. Un programa en `tinyS` comienza ejecutando el método `start`. Es importante remarcar que `start` no es una palabra reservada.

2.3. Miembros

El cuerpo de las definiciones de `class` consiste en una lista de definiciones de *atributos* para la clase definida por los `class`. `idclass` no puede tener dos atributos con nombre igual definidos en distintas definiciones.

En el cuerpo de los `impl` para un `class` estarán solamente los métodos de dicho `class`. En alguno de los `impl` debe estar el constructor que es un método con nombre un símbolo de puntuación punto o dot.

Un miembro de una clase es un *atributo*, un *método* o el constructor. Un *atributo* de un `class A` especifica una variable que es parte del estado de objetos de la clase `A`.

Un *método* de una clase `A` es un procedimiento que puede manipular variables y objetos de clase `A`.

Todos los *atributos* tienen un alcance **privado** que serán solo accesibles dentro de la clase salvo aquellos que tengan la palabra reservada `pub`. Una subclase no puede acceder a variables de una superclase a menos que sean públicas, tampoco puede redefinirlas. Es decir, la única forma de proveer acceso a los atributos privados de una clase es a través de sus métodos.

Todos los *métodos* tienen alcance **global**. Aquellos métodos que tengan antes de la palabra `fn` la palabra clave `st` serán métodos de clase.

Recordatorio: No puede accederse a una variable de instancia en un contexto estático.

```

class Mundo {
    Int a;
    pub Str b;
}

impl Mundo{
    .(){ a = 42;}
    fn Int get_a(){ ret a; }
    st fn imprimo_algo(){
        (IO.out_str("hola mundo"));
    }
}

...
class Prueba{
    ...
    Mundo c;

    ...
}

impl Prueba{

    .(){
        c = new Mundo();
        y = c.b; // Acceso correcto al atributo de la clase
        z = c.a; // Acceso incorrecto al atributo de la clase
        z = c.get_a(); // Acceso correcto al atributo de la clase
        (c.imprimo_algo());
    }
}

start{
}

```

Los **nombres** de los miembros (atributos o métodos) deben **comenzar** con **letra minúscula**. Dentro de una clase no se pueden definir métodos con el mismo nombre ni atributos con el mismo nombre. **Pero un método y un atributo pueden tener el mismo nombre.**

Veamos a continuación un fragmento del archivo `ejemplo.s` que muestra un ejemplo del uso de métodos y atributos.

```

class A {}
impl A {
    .() {}

    fn B retB(Int x) {
        if (x > 5) {
            ret new B(0);
        } else {
            ret new B(1);
        }
    }
}

```

```

class B {
    Bool x;

    pub Otra y;
}
impl B {
    .(Int x) {
        if (x > 5) {
            self.x = false;
        } else {
            self.x = true;
        }

        self.y = new Otra();
    }
}

class Otra {
    Str hola;
}
impl Otra {
    .() {
        self.hola = "hola";
    }
}

impl B{
    fn Bool getX() {
        ret self.x;
    }
}

start {
    A a;
    IO io;
    io = new IO();
    a = new A();
    (io.out_str(a.retB(10).y.hola));
}

```

No hay ningún mecanismo en `tinyS` para que los programadores desaloquen objetos. `tinyS` tiene gestión automática de memoria. Los objetos que no van a ser utilizados por el programa son desacolcados por el recolector de basura (*garbage collector*) en tiempo de ejecución. Que en este proyecto NO se implementa.

2.4. Herencia

La *herencia* de clases está permitida. La herencia se define de la siguiente manera:

```
class B : A {  
...  
}
```

Si tenemos una clase B que hereda de una clase A entonces B hereda los miembros de A. Se dice que la clase A es la superclase de B y B es la subclase de A. La semántica de B hereda de A es que B tiene todos los miembros definidos en A además de los suyos propios.

En el caso de que una superclase y una subclase definan el mismo nombre de método, entonces la definición dada en la subclase tiene prioridad.

Es **ilegal** redefinir los nombres de los atributos.

Además, para la seguridad de los tipos, es necesario imponer algunas restricciones sobre cómo se pueden redefinir los métodos. Si una definición de clase no especifica una clase principal, entonces la clase hereda de **Object** de forma predeterminada.

Una clase puede heredar solo de una sola clase (no se permite herencia múltiple). La relación superclase-subclase en las clases define un gráfico. Este gráfico NO puede contener ciclos. Por ejemplo, si B hereda de A, entonces A no debe heredar de B. Además, si B hereda de A, entonces A debe tener una definición de clase (class) en algún lugar del código fuente. Debido a que **tinyS** tiene una herencia única, se deduce que si se satisfacen ambas restricciones, el gráfico de herencia forma un árbol con **Object** como raíz.

2.5. Tipos

En **tinyS**, toda clase también es un **tipo**. Una *declaración de tipos* tiene la forma **A x**, donde x es una variable de tipo A. Toda variable debe tener una declaración de tipo en el punto en que aparece, por ejemplo, como parámetro formal en un método. Los tipos de todos los atributos deben estar también declarados.

La regla de tipo básica en **tinyS** es que si un método o variable espera un valor de tipo A, entonces cualquier valor del tipo B **se puede utilizar en su lugar**, siempre que A sea un **antepasado de B en la jerarquía de clases**.

El sistema de tipos garantiza en tiempo de compilación que la ejecución de un programa no resulte en un error de tipo en tiempo de ejecución. Utilizando los tipos declarados para los identificadores en el código fuente el análisis semántico infiere un tipo para toda expresión en el código fuente.

Es importante distinguir entre el tipo asignado por el verificador de tipos para una expresión en tiempo de compilación que llamaremos tipo estático de la expresión y los tipos o tipo que la expresión puede evaluar durante la ejecución que llamamos tipos dinámicos (leer teoría).

La distinción entre tipo estático y tipo dinámico es necesaria debido a que el verificador de tipos no puede, en tiempo de compilación, tener información perfecta sobre qué valor se va a computar en tiempo de ejecución. De hecho, el tipo estático y el tipo dinámico suelen ser distintos. Lo que necesitamos, sin embargo, es que los tipos estáticos del verificador de tipos sean correctos con respecto a los tipos dinámicos.

Todas las variables en **tinyS** se inicializan por defecto para contener valores del tipo apropiado. Se cuenta además con el tipo especial **void**.

2.6. Definición de Métodos

La definición de un método tiene la forma:

```
[ st ] fn [<type>] <idMétodo>([<parámetros formales>]){ <sentencias> }
```

En caso de que el método no contenga <type> se toma que el tipo de retorno es **void**.

La declaración de métodos de clase estará precedida por la palabra reservada `st`. Si `st` no aparece se trata de una declaración de método de instancia.

Puede haber cero o más parámetros formales. Los identificadores utilizados en la lista de parámetros formales deben ser distintos. El tipo del cuerpo del método debe ajustarse al tipo de retorno declarado. Dentro del método no puede haber identificadores con el mismo nombre de los parámetros formales del método. Cuando se invoca un método, los parámetros formales se vinculan a los argumentos reales y se evalúa la expresión; el valor resultante es el significado de la invocación del método. Un parámetro formal oculta cualquier definición de un atributo del mismo nombre. Es importante destacar que si el método retorna `void` no debe colocarse `elret void` en el cuerpo del método.

Para garantizar la seguridad de los tipos, existen restricciones sobre la redefinición de métodos heredados. La regla es simple: si una clase `B` hereda un método `f` de una superclase `A`, entonces `B` puede anular la definición heredada de `f` siempre que **el número de argumentos, los tipos de parámetros formales y el tipo de retorno sean exactamente iguales** en ambas definiciones. Un método de clase puede ocultarse (*hiding*) pero no anularse (*overriding*). Es decir, si una clase `B` hereda un método de clase `f` de una superclase `A`, entonces `B` puede definir un método de clase con el mismo nombre que `f` pero con diferente número de argumentos, tipos de parámetros formales o tipo de retorno. Recordar que los métodos de clase se enlazan a la clase y no a la instancia. Por lo tanto, el método de clase definido en `B` con el mismo nombre que el método de clase heredado de `A` no anula al método de clase heredado de `A` sino que lo oculta.

Para ver por qué es necesaria alguna restricción en la redefinición de métodos heredados, considere el siguiente ejemplo que retorna un error:

```
class A {  
}  
impl A { fn Int f(){ ret 1; } }  
class B : A {  
}  
impl B {fn Str f() { ret "1"; }} //error de redefinición de método heredado  
...
```

Tomemos un objeto `a` con tipo dinámico `A`. Entonces, `a.f() + 1` es una expresión bien formada con valor 2. Sin embargo, no es posible sustituir un valor de tipo `B` para `a`, ya que trataríamos de sumar un `Int` a un `Str`. Luego, si los métodos pudieran redefinirse de forma arbitraria, entonces las subclases no podrían extender de forma sencilla el comportamiento de sus superclases.

2.7. Definición del constructor

La definición del método `·` es llamado implícitamente y únicamente cuando un nuevo objeto es inicializado. La definición de `·` tiene la siguiente forma:

```
.(<Idclass> <id>,...,<Idclass> <id>) <bloque_método>;
```

2.8. Constructor

Una clase **siempre** debe tener definido uno y solo un método `·`.

`·` **puede** tener argumentos distintos en una subclase y una superclase.

Una expresión de inicialización provee acceso al inicializador de un tipo.

```
class UnaSubclase: UnaSuperClase {}  
impl UnaSubClase {
```

```
    .() {  
        // La inicialización de la subclase se coloca aquí  
    }  
}
```

Se llama a `·` al hacer **new** de una nueva instancia de clase. Tener en cuenta que `·` solo es utilizada al hacer el **new** no puede accederse en otro contexto.

2.9. Declaración de atributos

La definición de un atributo de clase tiene la siguiente forma:

```
[pub] <type> <id>;
```

Para el caso particular de los arreglos la definición de un atributo arreglo tiene el siguiente formato:

```
[pub] Array <type> <id>;
```

Los atributos no pueden ser redefinidos.

2.10. Declaración de parámetros Formales

En cuanto a la declaración de los parámetros formales esta deberá ser de la forma:

```
<type> <id>
```

En el caso de que el parámetro formal sea un arreglo la declaración es:

```
Array <type> <id>
```

3 Expresiones

Las expresiones en `tinyS` son:

3.1. Expresiones aritméticas, de comparación y operadores lógicos

- `tinyS` tiene cinco operaciones aritméticas binarias: `+`, `-`, `*`, `/`.

La sintaxis es `<expr1><op><expr2>`

- Para evaluar dicha expresión, primero se evalúa `<expr1>` y luego `<expr2>`. El resultado de la operación es el resultado de la expresión. Los tipos estáticos de las dos subexpresiones deben ser `Int`. El tipo estático de la expresión es `Int` si ambas expresiones son de tipo `Int`.

Ejemplo 3. Observar con atención ambos ejemplos.

```
start {
  Int a;
  Str c;
  Str b;
  b="3.0";
  a=10;
  c=a+a; //esto devuelve error de tipo
  IO.print_Str(c);}

```

```
start {
  Int a;
  Int c;
  Int b;
  b=3;
  a=10;
  c=a+b; //esto NO devuelve error de tipo
  IO.print_Int(c);}

```

Para el caso de `++` y `--` la sintaxis será `++<expr>` y `--<expr>`. Observar que la `<expr>` debe devolver un tipo `Int` y tiene precedencia más fuerte que los operadores unarios comunes (`+` y `-`).

- Para la operación `/` esta siempre devuelve la parte entera de la división. Por ejemplo, `3/2` devuelve `1` y `-3/2` devuelve `-1`.
- En `tinyS` las expresiones booleanas formadas con los operadores lógicos: `&&` (and), `||` (or) y `!` (not) y aquellas formuladas utilizando los operadores relacionales `<`, `<=`, `==`, `>=` y `!=`.
- La comparación por igual `==` es un caso especial. Si `<expr1>` o `<expr2>` tiene el tipo estático `Int`, `Bool` o `Str`, el otro debe tener el mismo tipo estático. Cualquier otro tipo puede compararse libremente. En objetos no básicos, la igualdad simplemente verifica la igualdad de la referencia (es decir, si las direcciones de memoria de los objetos son las mismas).

3.2. self

`self` es una referencia explícita a la instancia del tipo en el que ocurre.

`self.id`

Por ejemplo,

```

class UnaClase {
    Str greeting;
}

impl UnaClase {
    .(Str greeting) {
        self.greeting = greeting;
    }
}

...

```

3.3. Llamadas a métodos

Las llamadas a un método en `tinyS` suelen tener la siguiente forma:

```
<expr>.<id>(<expr>, ..., <expr>)
```

Tenga en cuenta que en `tinyS` los métodos siempre se llaman con los paréntesis aunque no tengan parámetros. Este es el caso de una llamada dentro de una expresión. La llamada como sentencia será vista en la Sección ??.

En el caso particular de los métodos de clase, la llamada a un método de clase tiene la siguiente forma:

```
<Idclass>.<id>(<expr>, ..., <expr>)
```

3.4. New

Una expresión `new` tiene la forma `new <type>([<parametros actuales>])`. Para el caso de los arreglos la expresión `new` queda de la siguiente forma `new Int[<expr>]`. Donde se especifica el tipo de los elementos del arreglo y el tamaño del mismo.

El valor es un objeto nuevo de la clase apropiada. Tenga en cuenta que el `new` realiza la reserva de espacio para la referencia al objeto y la inicialización de la clase `<type>`. Se llama al método `·` de la clase si es que existe.

El `new` para un objeto de tipo `Array` implica que se reserva el tamaño especificado por el valor de una expresión entera cuando se conozca y sus elementos son inicializados con el valor por defecto según su tipo primitivo de los elementos del arreglo. Además, el tipo declarado para los elementos debe coincidir con el tipo de la sentencia `new`.

Ejemplo 4. Por ejemplo,

```

fn void ejemplo(Int b){
    Array Int a;
    a = new Int[6+b];}

```

reserva el espacio de `6+b` elementos enteros y los inicializa a cada uno con cero. Luego, al acceder al elemento `a[2]` se obtiene 0 porque es el valor por defecto de un entero. Observar con atención que esto implica que el tamaño del arreglo no se conoce en tiempo de compilación sino que se conoce en tiempo de ejecución.

3.5. Acceso a Atributos

En `tinyS` el acceso a atributos privados de una clase fuera de la misma se realiza a través de los métodos que tenga definida dicha clase. Si el atributo está en la misma clase no hace falta poner ninguna expresión particular. **No es posible redefinir atributos sea cual sea su visibilidad.** Cuando se crea (`new`) un nuevo objeto de una clase, se deben inicializar todos los atributos heredados (públicos) y locales. Los atributos heredados se inicializan primero en orden de herencia comenzando con los atributos de la clase de ancestro más grande. Dentro de una clase determinada, los atributos se inicializan en el orden en que aparecen en el código fuente. Los atributos son privados salvo que se utilice la palabra `pub`. **Los atributos de superclases privados no son accesibles por su subclase** salvo a través de los métodos de la superclase.

4 Sentencias

Las sentencias en **tinyS** incluyen a las expresiones, asignaciones, declaraciones ya sea de constantes, atributos y métodos, bucles, bloques, condicionales. Se presenta a continuación una breve introducción de cada uno.

4.1. Asignación

La asignación tiene la forma (tenga en cuenta que una asignación es una expresión aunque la utilizaremos como sentencia):

```
<LadoIzquierdo>=<Expresión>;
```

El tipo del lado derecho (<Expresión>) de la asignación debe matchear con el tipo del lado izquierdo.

4.2. Sentencia Simple

tinyS realiza llamadas a métodos y otras expresiones como una sentencia de la siguiente forma:

```
(<Expresión>;
```

4.3. Bucle while

Un bucle **while** en **tinyS** respeta la siguiente forma:

```
while (<condición>) <sentencia>
```

La condición se evalúa **antes** de cada iteración del ciclo. Si la condición es **falsa**, el bucle termina. Si la condición es **verdadera**, se evalúa el cuerpo del bucle y el proceso se repite. El condicional debe tener tipo **Bool**.

4.4. Bucle for

Un bucle **for** en **tinyS** respeta la siguiente forma:

```
for (<type> <idVariable1> in <idVariable2>) do  
<sentencia>
```

Donde el `idVariable2` es de tipo **<iterator>** o sea un objeto que implementa la interfaz **iterador** y que se inicializa antes de la primera iteración del bucle. En particular para **tinyS** la clase **Array** implementa la interfaz **iterador**.

El bucle **for** se repite mientras el método **hasNext()** del iterador devuelva **true**. En cada iteración, el valor de la variable `<idVariable>` se asigna al resultado de la llamada al método **next()** del iterador. El tipo de la variable `<idVariable>` debe ser el mismo que el tipo de retorno del método **next()** del iterador. El iterador debe ser un objeto de una clase que implemente la interfaz **iterador**. El iterador se inicializa antes de la primera iteración del bucle. El iterador se puede modificar dentro del cuerpo del bucle, pero no es necesario hacerlo para que el bucle funcione correctamente. El bucle termina cuando el método **hasNext()** del iterador devuelve **false**.

Para la clase array la iteración comienza en el índice 0 y continúa hasta el índice del último elemento del arreglo. En cada iteración, el valor de la variable `<idVariable>` se asigna al valor del elemento del arreglo en el índice actual de la iteración. El bucle termina cuando se ha iterado sobre todos los elementos del arreglo.

4.5. Bloques

Un bloque en `tinyS` respeta la siguiente forma:

```
{ <sentencia>* }
```

Las sentencias se evalúan en orden de izquierda a derecha.

Los bloques en `tinyS` se diferenciarán entre bloque de método (aquellos que contienen declaraciones de variables) y bloque de sentencias (bloques dentro de `if-else`, `while` etc. que no tienen declaraciones de variables). Esto quedará más claro al observar la gramática.

4.6. Condicionales

La condición tiene la forma:

```
if <condición>
  <sentencia>
[else {<sentencia> //se ejecuta si la condición es falsa}]
```

La semántica de los condicionales es la utilizada normalmente. La condición se evalúa primero. Si es verdadera, se ejecuta lo que está dentro del `if`. Si la condición es falsa, se ejecuta el `else`. El `else` es opcional.

4.7. Retorno de método

El retorno de un método tiene la forma:

```
ret <expr>;
```

Solo se utiliza si el tipo de retorno es distinto de `void`. Además, el tipo de la expresión de retorno debe **matchear** con el tipo declarado de retorno de la función. Todo método tiene que tener al menos una sentencia de retorno si el tipo de retorno es distinto de `void`. **Observar que se debe asegurar que toda ejecución de un método con tipo de retorno distinto de void termine en una sentencia de retorno.**

5 clases Base y clases predefinidas

Ninguna de las clases o class base poseen método · aunque las clases base deben inicializarse correctamente de acuerdo a su semántica. `tinyS` cuenta con un pequeño conjunto de clases predefinidas y clases bases.

5.1. class predefinida Object

`Object`: La superclase o super eclassura de todas las clases o eclassuras de `tinyS` (al estilo de la clase `java.lang.Object` de Java). En `tinyS`, la clase `Object` no posee métodos ni atributos.

5.2. class Predefinida IO

`IO`: Contiene métodos útiles para realizar entrada/salida.

- `st fn out_str(Str s)`: imprime el argumento.
- `st fn out_int(Int i)`: imprime el argumento.
- `st fn out_bool(Bool b)`: imprime el argumento.
- `st fn out_array_int(Array Int a)`: imprime cada elemento del arreglo de tipo `Int`.
- `st fn out_array_str(Array Str a)`: imprime cada elemento del arreglo de tipo `Str`.
- `st fn out_array_bool(Array Bool a)`: imprime cada elemento del arreglo tipo `Bool`.
- `st fn Str in_str()`: lee una cadena de la entrada estándar, sin incluir un carácter de nueva línea.
- `st fn Int in_int()`: lee un entero de la entrada estándar.
- `st fn Bool in_bool()`: lee un booleano de la entrada estándar.

Tanto `Object` como `IO` pueden ser utilizadas sin necesidad de ser heredadas. Para el caso particular de `Array` en caso de que el mismo no se encuentre inicializado la llamada al método deberá imprimir la leyenda “**El arreglo no se encuentra inicializado**”. `tinyS` tiene otras cuatro clases básicas: (`Int`, `Bool`, `Str`).

5.3. Iterator

La clase `Iterator` es una interfaz que define los métodos necesarios para iterar sobre una colección de elementos. En particular, la clase `Array` implementa la interfaz `Iterator`. La clase `Iterator` define los siguientes métodos:

- `fn Bool hasNext()`: devuelve `true` si hay más elementos para iterar, de lo contrario devuelve `false`.

-
- `fn next_<type>()`: devuelve el siguiente elemento en la iteración. El tipo de retorno debe ser el mismo que el tipo de los elementos de la colección que se está iterando o sea el tipo de los elementos del arreglo. Para el ejemplo, si se está iterando sobre un arreglo de tipo `Int`, el método `next_int()` debe devolver un valor de tipo `Int`. Habrá un error de tipo si el tipo de retorno del método `next_<type>()` no coincide con el tipo de los elementos del array que se está iterando. Además hay un `next` para cada tipo de elemento del iterador, es decir, `fn next_int()` para el caso de un iterador de elementos de tipo entero `Int` y `fn next_str()` para el caso de un iterador con elementos de tipo `Str`.

La clase `Iterator` no puede ser instanciada directamente, sino que debe ser implementada por otras clases que proporcionen la funcionalidad de iteración. En el caso de la clase `Array`, la implementación de la interfaz `Iterator` permite iterar sobre los elementos del arreglo utilizando un bucle `for`. En `tinyS` solo la clase `Array` implementa la interfaz `Iterator`. **no se permite implementar la interfaz `Iterator` en otras clases distintas a `Array`.**

5.4. Array

La clase arreglo proporciona listas de tamaño estático de elementos de tipos primitivos (`Int`, `Bool`, `Str`). Tenga en cuenta que dos arreglos serán compatibles si el tipo de sus elementos es el mismo y el tamaño también. De otra manera serán incompatibles.

- `fn Int length()`.
- `length` devuelve la longitud del parámetro `self`.
- y los métodos de la interfaz `Iterator` (`hasNext()` y `next_<type>()`) para iterar sobre los elementos del arreglo.

El acceso a los elementos dentro de una clase arreglo se hace a través de su índice, `a[i]` donde `a` es un arreglo e `i` es el índice que ubica un elemento del arreglo. Tenga en cuenta que un arreglo comienza su índice en la posición 0 y el índice `i` no debe superar el tamaño del arreglo $0 \leq i \leq a.length() - 1$. Un arreglo no inicializado tiene tamaño 0. Es un error heredar o redefinir `Array`.

La creación de un arreglo se hace a través del `new`. En el momento de la creación los valores de los elementos del arreglo se inicializan a sus valores por defecto según su tipo primitivo.

- `new <tipo>[Expresion]`.

5.5. Int

La clase `Int` proporciona números enteros. No hay métodos especiales para `Int`. La inicialización predeterminada para las variables de tipo `Int` es 0 (no `void`). Es un error heredar o redefinir `Int`.

5.6. Str

La clase `Str` proporciona cadenas. Se definen los siguientes métodos:

- `fn Int length()`. `length` devuelve la longitud del parámetro `self`.
- `fn Str concat(Str s)`. El método `concat` devuelve la cadena formada al concatenar `s` después de `self`.

La inicialización predeterminada para las variables de tipo `Str` es `""` (nunca `void`!). Es un error heredar o redefinir `Str`.

5.7. Bool

La clase `Bool` brinda el `true` y `false`. La inicialización predeterminada para las variables de tipo `Bool` es `false` (no `void`). Es un error heredar o redefinir `Bool`.

`tinyS` sólo provee las características indicadas en la sección precedente. Características no indicadas, tales como por ejemplo, genericidad, anotaciones, `break`, clausura, selectores, expresión implícita de miembros, etc., no están soportadas por el lenguaje. Y no se deben hacer puesto que no serán válidas.

Las secciones restantes de este documento proporcionan una definición más formal de `tinyS`. La estructura léxica se abarca en la Sección ??, la gramática generadora de código fuente en la Sección ??.

6 Lexemas y tokens en tinyS

Los lexemas y tokens de `tinyS` además de las cosas que se deben ignorar en tiempo de compilación se detallan a continuación:

6.1. Espacios en blanco

El espacio en blanco consta de cualquier secuencia de caracteres: blancos (ascii 32), `\n` (nueva línea, ascii 10), `\r` (retorno de carro, ascii 13), `\t` (tabulación, ascii 9), `\v` (tabulación vertical, ascii 11). Tenga presente que para este compilador se limitará el uso de caracteres raros o poco frecuentes en la lengua castellana para evitar problemas de compilación y testeo del compilador.

6.2. Comentarios

`tinyS` tiene dos tipos de comentario:

- `//` Comentario simple: Todos los caracteres de desde `//` hasta el final de la línea son ignorados. Se deben poder utilizar en los comentarios caracteres como `#,$,%,&,@ , ¡, !, ?,` y la `ñ`.
- `/* ... */` comentario multilinea.

6.3. Identificadores

Los nombres de las variables locales, los parámetros formales de métodos, `self` y atributos de clase son identificadores. El identificador `self` puede ser referenciado, pero es un error asignar `self` o vincular `self` como un parámetro formal. También es ilegal tener atributos denominados `self`.

Las variables locales y los parámetros formales tienen alcance léxico. Los atributos públicos son visibles por las clases que heradan de ella y podrán accederse en código fuera de la clase como atributo de una instancia de dicha clase, a no ser que estén declarados como privados en cuyo no serán visibles por nadie salvo la clase donde están definidos. La vinculación de una referencia de identificador es el ámbito más interno que contiene una declaración para ese identificador, o al atributo del mismo nombre si no hay otra declaración. La excepción a esta regla es el identificador `self`, que está implícitamente vinculado en cada clase.

Los identificadores son cadenas (distintas de las palabras clave) que constan de letras, dígitos y el carácter de guión bajo o *underscore*.

Los identificadores de **tipo** (*class*) comienzan con una **letra mayúscula** y terminan con una **letra**, un **dígito** o guión bajo `_`.

Los identificadores de **objetos** comienzan con una **letra minúscula**. Los identificadores `self` y `void` son tratados especialmente por `tinyS` al igual que la palabra `Array`. Los símbolos sintácticos especiales (por ejemplo, paréntesis, corchetes, operador de asignación, etc.) se verán directamente en la gramática.

6.4. Palabras clave

Las palabras claves de `tinyS` son: `class`, `impl`, `else`, `false`, `if`, `ret`, `while`, `true`, `nil`, `false`, `new`, `fn`, `st`, `pub`, `self`, `div`, `for`, `in`. En `tinyS` las palabras clave son sensibles a mayúsculas y minúsculas.

6.5. Literales

Un literal es la representación en código fuente del valor de un tipo, como un número o un `Str` por ejemplo: `42` es un literal entero, `"hello world"` es un literal cadena y `true` es un literal booleano.

6.5.1. Literales Enteros

Los enteros son cadenas no vacías de dígitos del 0 al 9.

6.5.2. Literales cadenas

Las cadenas o `Str` están encerrados entre doble comillas `"..."`. El tamaño máximo no debe superar los 1024 caracteres. Una cadena no puede contener EOF. Una cadena no puede contener el null (carácter `\0`). Cualquier otro caracter puede incluirse en una cadena. Para simplificar no se aceptarán caracteres raros o poco usados como ideogramas chinos, Hangeul, símbolos griegos, etc. Keep it Simple!

Existen dentro de las cadenas caracteres de escape que se representan con una barra invertida `\` seguida de un caracter. Por ejemplo, `"\n"` representa el literal cadena que contiene un salto de línea. `"\"` es la comilla dentro de la cadena. `\"` es la barra invertida dentro de la cadena. `\t` es el tabulador dentro de la cadena. `\r` es el retorno de carro dentro de la cadena. Lo mismo para comilla simple.

6.5.3. Literal nulo

El literal nulo se representa mediante la palabra reservada `nil`. Por defecto las referencias en `tinyS` toman el valor `nil` si no son inicializadas de igual manera se utiliza `nil` para los arreglos no inicializados.

7 Gramática BNF extendida

A continuación se muestra la gramática libre de contexto en formato BNF extendido para **tinyS** .

Solo aquellos programas escritos sintácticamente en este formato serán válidos para el futuro compilador. Para que sea más amena la lectura de la gramática la misma está escrita en notación BNF extendida. Observe que en letra mecanográfica aparecen los tokens devueltos por el Analizador Léxico.

```
 $\langle program \rangle ::= \langle Lista-Definiciones \rangle \langle Start \rangle$   
 $\langle Start \rangle ::= \text{start} \langle Bloque-Método \rangle$   
 $\langle Lista-Definiciones \rangle ::= (\langle class \rangle \mid \langle Impl \rangle)^*$   
 $\langle class \rangle ::= \text{class idclass} \langle Herencia \rangle^? \{ \langle Atributo \rangle^* \}$   
 $\langle Impl \rangle ::= \text{impl idclass} \{ \langle Miembro \rangle^* \}$   
 $\langle Herencia \rangle ::= : \langle Tipo \rangle$   
 $\langle Miembro \rangle ::= \langle Método \rangle \mid \langle Constructor \rangle$   
 $\langle Constructor \rangle ::= . \langle Argumentos-Formales \rangle \langle Bloque-Método \rangle$   
 $\langle Atributo \rangle ::= \langle Visibilidad \rangle^? \langle Tipo \rangle \langle Lista-Declaración-Variables \rangle ;$   
 $\langle Método \rangle ::= \langle Forma-Método \rangle^? \text{fn} \langle Tipo-Método \rangle^? \text{idMetAt} \langle Argumentos-Formales \rangle \langle Bloque-Método \rangle$   
 $\langle Visibilidad \rangle ::= \text{pub}$   
 $\langle Forma-Método \rangle ::= \text{st}$   
 $\langle Bloque-Método \rangle ::= \{ \langle Decl-Var-Locales \rangle^* \langle Sentencia \rangle^* \}$   
 $\langle Decl-Var-Locales \rangle ::= \langle Tipo \rangle \langle Lista-Declaración-Variables \rangle ;$   
 $\langle Lista-Declaración-Variables \rangle ::= \text{idMetAt}$   
 $\qquad \qquad \qquad \mid \text{idMetAt} , \langle Lista-Declaración-Variables \rangle$   
 $\langle Argumentos-Formales \rangle ::= ( \langle Lista-Argumentos-Formales \rangle^? )$   
 $\langle Lista-Argumentos-Formales \rangle ::= \langle Argumento-Formal \rangle , \langle Lista-Argumentos-Formales \rangle$   
 $\qquad \qquad \qquad \mid \langle Argumento-Formal \rangle$   
 $\langle Argumento-Formal \rangle ::= \langle Tipo \rangle \text{idMetAt}$   
 $\langle Tipo-Método \rangle ::= \langle Tipo \rangle \mid \text{void}$   
 $\langle Tipo \rangle ::= \langle Tipo-Primitivo \rangle \mid \langle Tipo-Referencia \rangle \mid \langle Tipo-Arreglo \rangle$   
 $\langle Tipo-Primitivo \rangle ::= \text{Str} \mid \text{Bool} \mid \text{Int}$   
 $\langle Tipo-Referencia \rangle ::= \text{idclass}$   
 $\langle Tipo-Arreglo \rangle ::= \text{Array} \langle Tipo-Primitivo \rangle$ 
```

```

<Sentencia> ::= ;
              | <Asignación>;
              | <Sentencia-Simple>;
              | if (<Expresión>) <Sentencia>
              | if (<Expresión>) <Sentencia> else <Sentencia>
              | while ( <Expresión> ) <Sentencia>
              | for ( <Tipo-Primitivo> idMetAt in idMetAt ) <Sentencia>
              | <Bloque>
              | ret <Expresión>;

<Bloque> ::= { <Sentencia>* }

<Asignación> ::= <AccesoVar-Simple> = <Expresión>
                | <AccesoSelf-Simple> = <Expresión>

<AccesoVar-Simple> ::= id <Encadenado-Simple>* | id [ <Expresión> ]

<AccesoSelf-Simple> ::= self <Encadenado-Simple>*

<Encadenado-Simple> ::= . id

<Sentencia-Simple> ::= (<Expresión>)

<Expresión> ::= <ExpOr>

<ExpOr> ::= <ExpOr> || <ExpAnd> | <ExpAnd>

<ExpAnd> ::= <ExpAnd> && <ExpIgual> | <ExpIgual>

<ExpIgual> ::= <ExpIgual> <OpIgual> <ExpCompuesta> | <ExpCompuesta>

<ExpCompuesta> ::= <ExpAd> <OpCompuesto> <ExpAd> | <ExpAd>

<ExpAd> ::= <ExpAd> <OpAd> <ExpMul> | <ExpMul>

<ExpMul> ::= <ExpMul> <OpMul> <ExpUn> | <ExpUn>

<ExpUn> ::= <OpUnario> <ExpUn> | <Operando>

<OpIgual> ::= == | !=

<OpCompuesto> ::= < | > | <= | >=

<OpAd> ::= + | -

<OpUnario> ::= + | - | ! | ++ | -- | (Int)

<OpMul> ::= * | /

<Operando> ::= <Literal> | <Primario> <Encadenado>?

<Literal> ::= nil | true | false | intLiteral | StrLiteral

<Primario> ::= <ExpresionParentizada>
              | <AccesoSelf>
              | <AccesoVar>
              | <Llamada-Método>

```

$$\begin{aligned}
& | \langle \textit{Llamada-Método-Estático} \rangle \\
& | \langle \textit{Llamada-Conclassor} \rangle \\
\langle \textit{ExpresionParentizada} \rangle &::= (\langle \textit{Expresion} \rangle) \langle \textit{Encadenado} \rangle^? \\
\langle \textit{AccesoSelf} \rangle &::= \textbf{self} \langle \textit{Encadenado} \rangle^? \\
\langle \textit{AccesoVar} \rangle &::= \textbf{id} \langle \textit{Encadenado} \rangle^? \mid \textbf{id} [\langle \textit{Expresión} \rangle] \langle \textit{Encadenado} \rangle^? \\
\langle \textit{Llamada-Método} \rangle &::= \textbf{id} \langle \textit{Argumentos-Actuales} \rangle \langle \textit{Encadenado} \rangle^? \\
\langle \textit{Llamada-Método-Estático} \rangle &::= \textbf{idclass} . \langle \textit{Llamada-Método} \rangle \langle \textit{Encadenado} \rangle^? \\
\langle \textit{Llamada-Conclassor} \rangle &::= \textbf{new idclass} \langle \textit{Argumentos-Actuales} \rangle \langle \textit{Encadenado} \rangle^? \\
&\quad \mid \textbf{new} \langle \textit{Tipo-Primitivo} \rangle [\langle \textit{Expresion} \rangle] \\
\langle \textit{Argumentos-Actuales} \rangle &::= (\langle \textit{Lista-Expresiones} \rangle^?) \\
\langle \textit{Lista-Expresiones} \rangle &::= \langle \textit{Expresión} \rangle \mid \langle \textit{Expresión} \rangle , \langle \textit{Lista-Expresiones} \rangle \\
\langle \textit{Encadenado} \rangle &::= . \langle \textit{Llamada-Método-Encadenado} \rangle \\
&\quad \mid . \langle \textit{Acceso-Variable-Encadenado} \rangle \\
\langle \textit{Llamada-Método-Encadenado} \rangle &::= \textbf{id} \langle \textit{Argumentos-Actuales} \rangle \langle \textit{Encadenado} \rangle^? \\
\langle \textit{Acceso-Variable-Encadenado} \rangle &::= \textbf{id} \langle \textit{Encadenado} \rangle^? \\
&\quad \mid \textbf{id} [\langle \textit{Expresion} \rangle] \langle \textit{Encadenado} \rangle^?
\end{aligned}$$

7.1. Ejemplos de código en tinyS

A continuación un ejemplo de código escrito en tinyS .

```

start{
    Array Int a;
    a = new Int[5];
    a[0] = 0;
    a[1] = 2;
    a[2] = 4;
    a[3] = 6;
    a[4] = 8;
    for (Int i in a){
        IO.out_int(i);
        IO.out_str("\n");
    }
}

```

Otro ejemplo de código escrito en tinyS :

```

class Fibonacci {
    pub Int suma;
    pub Int i,j;
}

impl Fibonacci {

fn Int sucesion_fib(Int n){
    i=0; j=0; suma=0;
    while (i<= n){
        if (i==0){
            (imprimo_numero(i));
            (imprimo_sucesion(suma)); }
        else
            if(i==1){
                (imprimo_numero(i));
                suma=suma+i;
                (imprimo_sucesion(suma));}
            else{
                (imprimo_numero(i));
                suma=suma+j;
                j=suma;
                (imprimo_sucesion(suma));
            }

        (++i);
    }
    ret suma;
}

.(){
    i=0; // inicializo i
    j=0; // inicializo j
    suma=0; // inicializo suma
}

fn imprimo_numero(Int num){
    (IO.out_str("f_"));
    (IO.out_int(num));
    (IO.out_str("="));
}

fn imprimo_sucesion(Int s){
    // El valor es:
    (IO.out_int(s));
    (IO.out_str("\n"));
}

start{
    Fibonacci fib;
    Int n;
    fib = new Fibonacci();
    n = IO.in_int();
    (IO.out_int(fib.sucesion_fib(n)));
}

```